Audible Lava / DSO v4.10 — Rich Synth Palette: implementation companion

Sam Leventhal * November 2026 * companion to blob_synth_rich.scd, synth_palette.py, test_synth_palette.py and the v4.10 audible_lava.py

1. What this release does

v4.10 is an additive synthesis release. The observables, the routing matrix, the score governor, the pattern language and the melody engine are untouched; what changes is what is on the other end of each OSC (Open Sound Control) message. Every SynthDef gains a richer sibling built from the community plugin collection sc3-plugins (with stock-UGen fallbacks so the file compiles and sounds without them), a master space sits on the output bus and is driven by the governor's ScoreState every frame, and two things that were defined but never audible are wired in: the predicted-triad pre-sound (**\preSound**, defined since v4.2, spawned by nothing) and three controls of the sustained voice that no routing reaches (vowel formant, sub-octave weight, stereo width). The Python side chooses the palette by SynthDef name, so **--classic-synths** restores v4.9 exactly.

2. Observable → attribute → auditory effect (as of v4.9.6, traced)

The chain is: tracker observables per blob → routing.extract_source_value (normalised sources) → VoiceManager._compute_targets (base + sum of depth*curve(source), then the governor's summed layer) → first-order smoothing → /n_set on the slot's node every frame. Hits, pairs and melody notes are one-shot nodes spawned from the governor and the melody engine. The table is the complete default wiring; signs follow image coordinates (vy < 0 = rising).

Observable (source)	Normalisation	Destination(s)	What you hear
height (Y_NORM)	profile.y_norm, 1 = top (v4.10.1; was inverted, see §7)	PITCH +7 st; spawn note via y_to_midi (top = high)	register follows height; a blob climbs a fifth as it rises
persistence	/ (4*expected mean)	AMP +0.30 (sqrt); base_amp at spawn	clearer, more contrasted blobs are louder
vy (vertical velocity)	/ 3 px/frame → [–1,1]	WAVE_POS +0.5; CUTOFF –900 (rising brightens)	rising blobs brighten and move along the wavetable
area	/ blob_area_max (sqrt)	WAVE_POS +0.4	big blobs sit further into the bank
elongation	0..1	RES +0.6; DETUNE +0.45 (s-curve)	a stretched blob rings and beats
orientation	/ (π/2) → [–1,1]	PAN +1.0 (plus x at spawn)	tilt pans; blobAngle pad pans with it
speed	/ 3 → 0..1	CUTOFF +1800 (sqrt); DETUNE +0.35	fast blobs are bright and wide
d_area (growth)	/ 0.5 per s → [–1,1]	DRIVE +0.6; AMP +0.12	blooming blobs swell and fold harmonics
d_persistence	/ 0.5 per s	NOISE –0.6 (dissolving → air)	a dissolving blob breathes
accel (|ay|)	/ 0.3	SHIMMER +0.7 (sqrt)	turning blobs get vibrato + tremolo
direction change	event	attack/release preset (unless edited)	rising = fast attack, falling = slow
low + falling + expanding	gesture, hysteresis, <=2 voices, <=6 s	\bassStutter replaces the slot's node	sub-bass chop; rate from growth, wobble from fall speed
birth / split / merge / death	tracker events	\blobStrike on birth; silent split; merge glides and fades	transients mark births only

Observable (source)	Normalisation	Destination(s)	What you hear
TENSION (governor)	0..1 composite	DRIVE +0.55; CUTOFF +2200; summed layer: detune/noise/drive	tense passages saturate and brighten
BEAT_PHASE	0..1	AMP –0.28; CUTOFF –1400 (s-curve)	a periodic accent on the sustained voices
PROMINENCE_RANK	1 = head	AMP +0.35 (sqrt); CUTOFF +1500; mixer scaling	heads come forward, ornaments recede
PHRASE_POSITION	0 boundary → 1	WAVE_POS +0.45	timbre drifts across the phrase
SURPRISAL	nats / 6	SHIMMER +0.6 (sqrt)	unexpected transitions shimmer
AMBIGUITY	cond. entropy, 0..1	NOISE +0.4; pitch field adds tones	uncertain harmony = more air, more pitch classes
DEPTH (merge tree)	0 root → 1 leaf	PITCH +12	deep leaves an octave up (0 when unplaced)
pitch_field + gravitation	Tonnetz walker	freq pulled toward field (weaker for heads)	voices settle into the current triad
boundary (strength > 0.6)	hazard event	attack reset to <= 0.45 s	a phrase start re-articulates
pattern layer	E(k,n) per blob; k from area, n from its own period, swing from elongation	\mpeHit one-shots, timed bundles	ornamental hits, louder for low-rank blobs
paired voice (top third by prominence)	E(beats, 2*beats), tension fills gaps	\mpeHit an octave below the sustained note	heads are articulated: tap, rest, tap
hazard onsets (melody)	h = h0(1 + 4*speed + 3*accel), <= 1/s	\melodyHit; pitch from height band with hysteresis; dur in beats from area; legato if expanding	a slow blob speaks every ~4 s; motion clusters notes
predicted_triad, harmony_posterior_other	harmony beam posterior	**nothing (until v4.10)**	—

3. Plugins reviewed

SC_Playground.scd demonstrates Gendy3 (core), JPverb and Greyhole (DEINDUGens), OteyPiano, SmoothDecimator (BerlachUGens), VSTPlugin hosting ValhallaSupermassive, and Pbind/Pseq/Pwhite sequencing. Against the sc3-plugins 3.13 inventory, the following were selected for the project on the criterion *does this map an observable or a governor quantity onto something a listener can hear?*

UGen (package)	What it is	Used for
SVF (BlackrainUGens)	state-variable filter with weighted LP/BP/HP outputs	the fmode morph as one filter pass (classic used three filters + SelectX; the hits file notes the header said SVF but the code was not)
MoogLadder	4-pole ladder	\bassStutterRich wobble
DWGPlucked / DWGBowed (DWGUGens)	digital-waveguide strings	ornamental hits plucked; melody notes bowed when a blob expands, plucked when it shrinks; bow pressure from speed
OteyPiano / MdaPiano	physical piano / mda piano	the paired head voice an octave below the sustained note
MembraneCircle (MembraneUGens)	2-D waveguide drum	births: blob area → tension (size), elongation → loss (ring)
Streson (BhobUGens)	string resonator	the `noise` control: air that rings at the fundamental
Gendy1 (core)	dynamic stochastic synthesis (Xenakis)	\blobAngleRich pad wanders ±0.5 % around 1.5× the voice pitch
BlitB3Square / BlitB3Saw	band-limited oscillators	sub-octave blend of the voice; bass stack
SmoothDecimator	smoothed rate/bit crusher	bass grit that follows stutterRate

UGen (package)	What it is	Used for
JPverb + Greyhole	lush reverb; metallic drifting delay/reverb	the master space (ScoreState-driven)
VSTPlugin (IEM)	VST2/VST3 host UGen	optional master; params p0..p7 reachable by /n_set
SensoryDissonance, Chromagram, KeyTrack	analysis UGens	not used; candidates for closing the loop (measured roughness vs roughness_target) — would need SC→Python OSC, which the design currently avoids

Pbind. The pattern system schedules on sclang's TempoClock. The project's timing principle is that Python owns time (elastic, projective clock; hits as timestamped bundles; no SC TempoClock), so Pbind is kept out of the live path on purpose. It is used in blob_synth_rich.scd only as an *audition* tool (~audition) for listening to the hit family offline, as the playground did with OteyPiano. **Gendy3 note:** the playground calls Gendy3 with minfreq/maxfreq, which are Gendy1's arguments (Gendy3 takes a single freq); the pad here uses Gendy1.

4. The new SynthDefs

Each is a drop-in sibling: it accepts exactly the controls Python already sends to its classic twin, so no /s_new or /n_set changes, plus optional extras defaulting to off. Every plugin is reached through \Name.asClass so the file compiles without sc3-plugins; the post window lists what was found.

SynthDef	Replaces	Engine	Extras (sender)
\mpeVoiceRich	\mpeVoice	VOsc ×3 unison → fold+tanh drive → SVF morph → Streson air → chorus	sub ← area, formant ← height (oh→ah→ee), chorus ← |vx| (VoiceExtras, every frame, change-gated)
\mpeHitRich	\mpeHit	model 0 wavetable stack / 1 DWGPlucked / 2 OteyPiano→MdaPiano→Klank, then drive, SVF morph, air	model ← layer (ornament 1, pair 2) via RichPatternScheduler
\melodyHitRich	\melodyHit	model –1 auto (release >= 0.8 → bowed, else plucked) / 0 / 1 / 2	model, bow ← d_area, speed (RichMelodyEngine)
\bassStutterRich	\bassStutter	BlitB3 saw+square → MoogLadder → stutter latch → SmoothDecimator	crush (default 0.5)
\blobStrikeRich	\blobStrike	MembraneCircle + Klank, impulse excited	—
\blobAngleRich	\blobAngle	Gendy1 band + sine pair, LPF	—
\preSoundRich	\preSound	3 partials + slow drift + faint air	driven by PreSoundLayer
\lavaMaster	(new)	tail of group 0: In.ar(0) → drive → JPverb + Greyhole → Limiter → ReplaceOut	wet, t60, size, damp, modDepth, holeMix, holeFb, highcut, drive, t_bloom, bloomAmt, bypass
\lavaMasterVST	(optional)	same insert around VSTPlugin.ar	wet, bypass, p0..p7

5. The master space mapping (synth_palette.master_params_from_state)

The insert works without any routing change because node order is execution order: Python spawns every voice, hit and pad at the head of group 0, the master is added at the tail, reads the summed bus 0 and replaces it. Python spawns it (node 900) after VoiceManager, because VoiceManager's /g_freeAll 0 at startup frees anything spawned earlier from the language. All parameters are lagged in SC (1.5–4 s) so the room moves at the lamp's timescale, not the frame rate. `depth` scales every departure from the neutral room.

ScoreState field	Control	Range	Reading
ambiguity	t60	1.5 → 8.5 s	harmonic ambiguity is haze: the tail lengthens as the field widens
1 – tension	damp	0.80 → 0.15	tension keeps the tail bright; release darkens it
phrase_position	wet	0.22 → 0.55	the phrase dissolves into its boundary
surprisal / 6	holeFb	0.25 → 0.85	unexpected transitions make the metallic drift feed back

ScoreState field	Control	Range	Reading
tension	holeMix, drive	0.04 → 0.34; 0 → 0.45	tense passages get the Greyhole shimmer and saturation
1 – kuramoto	modDepth	0.90 → 0.15	incoherent blob motion = a drifting, chorused space; locked blobs = still
count_smooth / 6	size	3.2 → 1.2	an emptying lamp opens the room
register_center	highcut	2.5 → 10 kHz	the tail follows where the mass sits
boundary (strength >= 0.3)	t_bloom, bloomAmt	one trigger	a 4 s swell of feedback and wet level at each grouping boundary

6. Wiring and operation

File	Status	Role
blob_synth_rich.scd	NEW	all rich SynthDefs, master, VST master, plugin check, manual tests, Pbind audition
synth_palette.py	NEW	Palette (CLASSIC/RICH), apply_palette, RichPatternScheduler, RichMelodyEngine, VoiceExtras, MasterSpace, PreSoundLayer, SynthPaletteRig/install
test_synth_palette.py	NEW	11 tests, no SuperCollider/camera/pygame (Results: 11 passed)
audible_lava.py	MODIFIED	imports synth_palette; melody engine via make_melody_engine; install() after the governor; rig.update() after the melody layer; 'x' bypass key; HUD line; close at exit; flags --classic-synths --no-space --no-presound
routing.py	MODIFIED (v4.10.1)	Source.Y_NORM returns profile.y_norm as is (1 = top); test_v4101_ynorm.py
build_companion_v410.py	NEW	this document

SuperCollider load order: blob_synth.scd → blob_synth_hits.scd → melody_hit.scd → blob_synth_rich.scd, then **python audible_lava.py**. Node ids: master 900, pre-sound 901–903 (VoiceManager starts at 1000). Post-window line to expect: *blob_synth_rich.scd: plugins found …* followed by *Audible Lava v4.10: rich palette ready (…)*. On the HUD the second status line reads SYNTH rich / space wet t60 size fb / pre=<triad>@amp. Press x to A/B the master insert; --classic-synths reproduces v4.9 bit for bit.

```
# minimal integration (what the v4.10 audible_lava.py does)
melody = synth_palette.make_melody_engine(voices, profile, palette, enabled=...)
rig    = synth_palette.install(voices, profile, governor=governor, melody=melody,
                               palette="rich", master=True, presound=True)
...   # per frame, after voices.process_frame() and melody.process_frame():
rig.update(voices.score_state, tracked)
...   # at exit, after release_all() and its 3.5 s settle:
rig.close()
```

7. Findings while tracing

Y_NORM is inverted relative to its documentation. lamp_profile.LampProfile.y_norm already returns 1 = top of lamp; routing.extract_source_value(Source.Y_NORM) returns 1 − profile.y_norm, i.e. 1 at the *bottom*. The spawn pitch is right (voice_manager._init_voice_from_blob inverts again and y_to_midi inverts a third time, so top = high), but the continuous Y_NORM → PITCH +7 routing lifts blobs near the bottom and drops a blob by a fifth as it rises — the opposite of routing.py's 'a blob born at the bottom and rising to the top climbs a fifth'. The melody engine reads profile.y_norm directly and is correct, which is why melody and drone registers can disagree in the same blob. One-line fix: return profile.y_norm(...) in extract_source_value — **applied in v4.10.1** (routing.py, regression test test_v4101_ynorm.py). No routing depth was retuned: the +7 st now lands on top blobs instead of bottom ones, so the sustained register sits a fifth higher at the top of the lamp than before and a fifth lower at the bottom.

\preSound was never spawned. Defined in blob_synth_hits.scd (v4.2) with harmony_posterior_other computed every frame; no Python module referenced it. PreSoundLayer wires it; pre-sound is silent when the prediction equals the current triad, scaled by VoiceManager.score_gain.

Playground Gendy3 arguments. Gendy3.ar has a single *freq*; minfreq/maxfreq belong to Gendy1. **'SVF' in the mpeVoice header** was RLPF/BPF/RHPF + SelectX in code (noted in the hits file); the rich voice is a real SVF when the plugin is present.

8. Risks and tradeoffs

Risk	Assessment / mitigation
CPU on Raspberry Pi 5	JPverb + Greyhole together cost roughly 3–6 % of a Pi 5 core at 48 kHz; DWG strings and OteyPiano are per-hit nodes that free themselves, so cost scales with hit density (density_ceiling caps it). MASTER_DEPTH and --no-space / --classic-synths are the fallbacks; measure with s.avgCPU before a show.
sc3-plugins on the Pi	The release ships Linux builds; if a UGen is missing the fallbacks are used silently and the post window says which. Keyword-argument names for OteyPiano (vel) follow the help file; a wrong keyword prints a warning and is ignored rather than failing.
ReplaceOut on bus 0	Correct only because every SynthDef writes to bus 0 at the head of group 0 (verified for all seven families). Anything added later with Out.ar to another bus, or added at the tail, bypasses or precedes the master; keep new SynthDefs on `out = 0` and spawned with addAction 0.
/g_freeAll at startup	Python frees the language-spawned master; synth_palette spawns its own after VoiceManager. ~spawnMaster is for language-only sessions.
Loudness	The master has a Limiter (0.92); reverb wet never exceeds 0.9 and pre-sound 0.10 at full posterior mass. The bloom adds <= +0.25 wet for 4 s. Gain-stage with x (bypass) before trusting a venue level.
Timing	Nothing here adds latency: master params are lagged kr controls, hit extras ride the existing timestamped bundles, melody extras are appended to the existing /s_new.
Crossfade discipline	No fades or handovers were added. Palette switches happen at spawn time (a slot's next node), the master's lags are smoothing of governed quantities, and the pre-sound gain is the posterior itself.

9. Tests

test_synth_palette.py covers: palette swap and restore (scheduler marks preserved); per-layer hit model with governor params winning; melody model/bow from d_area and speed, appended only to note sends; voice extras mapping, skipping bass/zombie/idle slots and change-gating; master mapping monotonicity, depth blend and ranges; master spawn at the tail, bloom gating on strength, bypass, free; triad parsing and register placement; pre-sound amplitude rules (no prediction, prediction = current, posterior × score_gain, silence on arrival); and the end-to-end rig for both palettes. Run: **python test_synth_palette.py**.